Taller práctico proyecto integrador: calidad de datos para la retención estudiantil

Análisis de deserción con datos institucionales sintéticos

1. Contexto y objetivo

Una institución de educación superior quiere entender qué factores están asociados a la deserción estudiantil, para diseñar estrategias de retención tempranas. ¿Qué características presentan los estudiantes con mayor riesgo de abandonar sus estudios?

Antes de responder cualquier pregunta, el equipo de datos debe garantizar que la información sea confiable. Un dato mal registrado puede cambiar la conclusión: si los promedios están en escala de 0 a 100 y no se corrigen, el promedio de la institución parece más alto de lo que es. Si una columna numérica llega como texto, las estadísticas no se pueden calcular. Por eso este taller empieza por la calidad y termina en las preguntas de negocio.

Al terminar el taller, debes ser capaz de:

- Documentar un dataset: describir cada columna, su tipo y su papel en el análisis.
- Diagnosticar problemas de calidad de distinto tipo (nulos reales, faltantes disfrazados, valores imposibles, duplicados, errores tipográficos y fechas en formatos mixtos) y distinguirlos entre sí.
- Tomar decisiones de limpieza justificadas, registrándolas para que otra persona pueda revisarlas.
- Calcular tasas y promedios por grupo, y reconocer cuándo una diferencia no es confiable porque el grupo es pequeño.
- Distinguir asociación de causalidad y explicar por qué una diferencia observada no prueba un efecto.
- Identificar variables que no deberían usarse para predecir, y valorar el riesgo de usar datos sensibles en decisiones institucionales.

2. Cómo usar esta guía

Cada método nuevo aparece con la misma secuencia: qué hace, cómo se escribe, un ejemplo con su salida, cómo leer el resultado, errores frecuentes y, al final, tu turno con el dataset real. Lee el ejemplo antes de escribir el código.

Los datos de la muestra son inventados y no corresponden al dataset real. Tienen seis filas y ocho columnas, con los mismos tipos de problema que el dataset real: un nulo en promedio, un promedio de 45, un duplicado, asistencias con símbolo de porcentaje o con guion, una variante de escritura del programa y una jornada en minúsculas. No tienes que escribir la muestra: sirve para entender cada método antes de aplicarlo.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

	id	programa	jornada	promedio	asistencia	trabaja	horas	estado
0	E1	Ingeniería de Sistemas	Diurna	4.2	91%	no	0	activo
1	E2	Psicologia	nocturna	3.1	-	sí	20	desertor
2	E3	Derecho	Diurna	45.0	78%	no	0	activo
3	E4	Ingeniería de Sistemas	Nocturna	NaN	86%	sí	35	desertor
4	E2	Psicologia	nocturna	3.1	-	sí	20	desertor
5	E5	Enfermería	Fin de semana	3.8	95%	no	-	graduado

3. Paso 0: ambiente virtual

Un ambiente virtual guarda las librerías de este proyecto por separado de las demás. Abre una terminal dentro de la carpeta del proyecto.

Crear el ambiente:

```
# Mac / Linux
python3 -m venv venv

# Windows
python -m venv venv
```

Activar el ambiente:

```
# Windows (PowerShell)
venv\Scripts\Activate.ps1

# Mac / Linux
source venv/bin/activate
```

Sabrás que está activo porque aparece (venv) al inicio de la línea de la terminal.

Instalar las librerías:

```
pip install -r requirements.txt
```

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Ejecutar el script:

```
python analisis_desercion.py
```

4. Mapa del proyecto

El trabajo se hace en un solo archivo, analisis_desercion.py. Tiene una función por paso de esta guía, y cada función tiene un TODO con preguntas que debes responder antes de programar.

```
proyecto/
├── analisis_desercion.py
├── requirements.txt
├── LEEME_ambiente.txt
└── Data/
    ├── raw/
    │   └── datos_estudiantes.csv
    └── processed/
        ├── dataset_limpio.csv
        ├── decisiones.csv
        └── graficos/
```

No modifiques Data/raw. El script siempre carga el dataset original desde ahí.

4.1 Registrar cada decisión con registrar()

El script incluye la función registrar(), que ya está escrita. Cada vez que tomas una decisión de limpieza, la llamas con cinco datos: el paso del taller, la columna afectada, la acción que hiciste, el número de filas afectadas y la justificación. Al final, todas las decisiones se guardan en un solo archivo, decisiones.csv, que permite revisar tu proceso completo.

```
registrar("paso", "columna", "acción", filas, "justificación")
```

Ejemplo de lo que se guarda: paso faltantes, columna promedio_acumulado, acción eliminar filas, 298 filas afectadas, justificación: una nota no se puede inventar. Escribe justificaciones que expliquen por qué tomaste la decisión, no solo qué hiciste.

Errores frecuentes:

- Pasar los argumentos en otro orden: registrar() espera el paso antes que la columna.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

- Registrar la acción sin contar las filas afectadas, o poner 0 cuando sí hubo cambios.

5. Paso 1: diccionario de datos y primera mirada

Antes de tocar un solo dato, debes entender qué significa cada columna. Un diccionario de datos es una tabla que describe cada columna: qué contiene, qué tipo de dato es y qué papel cumple en el análisis.

Para cada columna escribe tres cosas:

- Tipo: numérica (se puede sumar o promediar), categórica (valores de una lista), de texto o de fecha.
- Descripción: qué significa esa columna para la institución, con tus palabras.
- Rol: identificación, explicativa o resultado. Identificación sirve para distinguir a un estudiante y no debe usarse en un promedio. Explicativa describe al estudiante y puede explicar la deserción. Resultado es lo que queremos entender.

Un ejemplo de una fila llena sería: beca | categórica | indica si el estudiante recibe una beca (sí o no) | explicativa. Otro: estado | categórica | situación del estudiante: activo, graduado o desertor | resultado.

Completa la tabla con las 23 columnas del dataset. Revisa el archivo antes de llenarla: algunas columnas tienen valores que no esperarías ver en su tipo.

Columna	Tipo	Descripción	Rol
id_estudiante			
programa			
facultad			
jornada			
semestre			
promedio_acumulado			
creditos_aprobados			
materias_reprobadas			
asistencia_pct			
sesiones_tutoria			
cambios_programa			
estrato			
beca			
trabaja			
horas_trabajo_semana			
distancia_km			
edad			
genero			
ciudad_procedencia			
tipo_colegio			
fecha_matricula			
estado			
motivo_retiro			

Después de llenar la tabla, ejecuta el script para revisar la estructura del dataset. El paso 1 del script muestra la forma, los tipos y las estadísticas.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

5.1 shape: tamaño del dataset

shape devuelve dos números: el número de filas y el número de columnas. Se escribe sin paréntesis, porque es un atributo del dataframe y no una función.

```
print(df.shape)
```

Con el ejemplo de la muestra, la salida es:

```
(6, 8)
```

Cómo leer el resultado: el primer número son las filas y el segundo las columnas. En la muestra, (6, 8) significa seis filas y ocho columnas.

Errores frecuentes:

- Escribir df.shape() con paréntesis produce un error, porque shape no es una función.

Tu turno: compara las filas y columnas del archivo con las que describe el enunciado. Si no coinciden, ¿qué crees que pasó al leer el archivo?

5.2 info(): tipos de datos y nulos

info() muestra un resumen de cada columna: cuántos valores no nulos tiene y qué tipo de dato es. Imprime el resultado directamente, así que no necesitas print().

```
df.info()
```

Con el ejemplo de la muestra, la salida es:

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id          6 non-null      str
 1   programa    6 non-null      str
 2   jornada     6 non-null      str
 3   promedio    5 non-null      float64
 4   asistencia  6 non-null      str
 5   trabaja     6 non-null      str
 6   horas       6 non-null      str
 7   estado      6 non-null      str
dtypes: float64(1), str(7)
memory usage: 516.0 bytes
```

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Cómo leer el resultado: la columna Non-Null Count indica cuántos valores tienen contenido. Si una columna tiene menos valores no nulos que el total de filas, tiene celdas vacías. En Dtype, float64 son decimales, int64 son enteros y str es texto. Observa que asistencia y horas aparecen como str aunque deberían ser números: alguien escribió símbolos o guiones dentro de ellas.

Errores frecuentes:

- Confundir los no nulos con las filas: un valor no nulo es una celda con contenido, no una fila completa.
- Asumir que una columna de números es numérica. Revisa siempre el tipo con info().

Tu turno: identifica las columnas que deberían ser numéricas y aparecen como texto. Anótalas, porque las convertirás en el paso 3.

5.3 describe(): estadísticas de columnas numéricas

describe() calcula estadísticas básicas de una columna numérica: cuántos valores tiene, promedio, dispersión y cuartiles.

```
print(df["promedio"].describe())
```

Con el ejemplo de la muestra, la salida es:

```
 count     5.000000
mean     11.840000
std      18.543004
min       3.100000
25%       3.100000
50%       3.800000
75%       4.200000
max      45.000000
Name: promedio, dtype: float64
```

Cómo leer el resultado: count es la cantidad de valores no nulos, y mean es el promedio. std es la desviación estándar, que mide qué tan separados están los valores del promedio. 25%, 50% y 75% son los cuartiles, y 50% es la mediana. min y max muestran el rango: si el máximo es imposible, hay un error de registro. En la muestra, el máximo de 45 en promedio revela una escala distinta.

Errores frecuentes:

- describe() no sirve para columnas de texto. Si la columna aparece como texto, describe() no la incluye.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

- Usar el promedio como medida central cuando hay valores extremos. La mediana se afecta menos.

Tu turno: busca en describe() un mínimo o un máximo que no tenga sentido para su columna. Anota cuál y por qué.

5.4 value_counts(): revisar valores de una categoría

value_counts() cuenta cuántas veces aparece cada valor distinto de una columna, de mayor a menor. Por defecto no cuenta los nulos.

```
print(df["programa"].value_counts())
```

Con el ejemplo de la muestra, la salida es:

```
 programa
Psicologia                 2
Ingeniería de Sistemas     1
Derecho                    1
Ingeniería de Sistemas     1
Enfermería                 1
Name: count, dtype: int64
```

Cómo leer el resultado: cada fila de la salida es un valor distinto y el número es cuántas veces aparece. Si el mismo programa aparece con dos escrituras distintas, son filas separadas. En la muestra, 'Ingeniería de Sistemas' aparece dos veces porque una de las escrituras tiene un espacio al final.

Errores frecuentes:

- Creer que dos valores que se ven iguales son iguales para Python. Un espacio o una tilde cambian el valor.

Tu turno: revisa value_counts() de programa, jornada, genero, ciudad_procedencia y estado. Escribe qué valores parecen variantes de otros.

6. Paso 2: auditoría de calidad

Auditar significa buscar problemas sin corregirlos todavía. Hay seis tipos que debes distinguir, porque cada uno se trata de forma diferente:

- Nulo real: una celda vacía. Pandas lo detecta con isnull().
- Faltante disfrazado: un valor que significa 'sin dato' pero parece un dato, como '-' o 999. Pandas no lo detecta solo.
- Valor imposible: un dato que no puede existir, como una edad de 150 años o un promedio de 45 en una escala de 0 a 5.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

- Duplicado: una fila idéntica a otra en todas sus columnas.
- Error tipográfico: el mismo valor escrito de forma distinta.
- Formato inconsistente: una misma columna con fechas en dos formatos, o números con símbolos.

Un mismo problema puede tener más de un tipo. Por ejemplo, un promedio de 45 es un valor imposible, pero también puede ser un error de formato si la escala cambió.

6.1 isnull().sum(): contar nulos reales

isnull() revisa cada celda y devuelve True si está vacía. sum() suma esos valores por columna. Como True vale 1 y False vale 0, la suma cuenta los nulos.

```
print(df.isnull().sum())
```

Con el ejemplo de la muestra, la salida es:

```
 id            0
programa       0
jornada        0
promedio       1
asistencia     0
trabaja        0
horas          0
estado         0
dtype: int64
```

Cómo leer el resultado: cada número es la cantidad de nulos de esa columna. En la muestra, solo promedio tiene un nulo.

Errores frecuentes:

- Comparar con == None o == np.nan no sirve para encontrar nulos. Siempre usa isnull().
- Un nulo no es lo mismo que el texto 'nan' o 'NA' escrito a mano. Revisa las dos cosas.

Tu turno: cuenta los nulos de todas las columnas. Separa los que parecen errores de los que pueden ser esperados. ¿En qué columna un nulo sería esperado?

6.2 Contar faltantes disfrazados

Un faltante disfrazado es un valor que parece dato pero significa que falta. Para contarlo, compara la columna con ese valor usando ==. El resultado es True o False por fila, y sum() cuenta los True.

```
print((df["asistencia"] == "-").sum())
```

Cómo leer el resultado: en la muestra, la asistencia con guion aparece dos veces. Pandas no la considera nula, porque el guion es texto. Por eso no basta con isnull().

Errores frecuentes:

- Usar == con números en texto: el valor -1 o 999 en una columna de texto no es igual al número.
- Olvidar que la comparación es sensible a mayúsculas: 'Unknown' y 'unknown' no son iguales.

Tu turno: cuenta los guiones en asistencia_pct y horas_trabajo_semana, y los valores 999 en distancia_km. Anota cuántos encontraste.

6.3 str.contains(): buscar un símbolo dentro de un texto

str.contains('símbolo', regex=False) revisa cada texto y devuelve True si contiene ese símbolo. regex=False indica que el símbolo se busca literal, sin interpretar caracteres especiales.

```
print(df["asistencia"].str.contains("%", regex=False).sum())
```

Con el ejemplo de la muestra, la salida es:

```
4
```

Cómo leer el resultado: el número es cuántas celdas tienen el símbolo. En la muestra, cuatro asistencias tienen el signo %. Esa es la señal de que la columna es texto y no número.

Errores frecuentes:

- Olvidar .str antes de contains: la columna completa no tiene ese método.
- Usar el símbolo sin regex=False cuando tiene significado especial, como el punto.

Tu turno: cuenta cuántas celdas de asistencia_pct tienen el signo %.

6.4 Contar duplicados

duplicated() devuelve True en cada fila que es igual a una fila anterior en todas sus columnas. La primera aparición queda en False.

```
print(df.duplicated())
```

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Con el ejemplo de la muestra, la salida es:

```
 0    False
1    False
2    False
3    False
4     True
5    False
dtype: bool
```

Cómo leer el resultado: la fila 4 aparece como True porque es idéntica a la fila 1. Si una sola celda cambia, por ejemplo un espacio, la fila ya no es igual y no se marca.

Errores frecuentes:

- duplicated() solo compara contra filas anteriores. Si una fila aparece dos veces, la marcada es la segunda.

Tu turno: cuenta las filas duplicadas. Antes de eliminarlas, responde: ¿podrían existir dos estudiantes distintos con todos los datos iguales? ¿Qué columna los diferenciaría?

6.5 Valores imposibles con condiciones

Una condición con dos reglas se escribe con & (y) o | (o), y cada condición va entre paréntesis. Por ejemplo, para marcar edades fuera de un rango, comparas dos veces y combinas los resultados con |.

```
df = df[df["promedio"] <= 5]
```

Cómo leer el resultado: cada paréntesis es una condición separada. Si olvidas los paréntesis, Python evalúa los operadores en otro orden y el resultado es incorrecto o produce un error.

Errores frecuentes:

- Usar and u or en lugar de & y |. Con columnas de Pandas no funcionan.
- Olvidar que un nulo no cumple ninguna condición: una edad vacía nunca es menor que 15, ni mayor que 60.

Tu turno: escribe la condición para marcar edades fuera de 15 a 60, y otra para promedios mayores a 5. Cuenta cuántas filas cumplen cada una.

7. Paso 3: convertir columnas de texto a números

Algunas columnas numéricas llegan como texto porque tienen símbolos o guiones. Para usarlas en promedios y tasas, hay que quitar esos símbolos y convertirlas. El proceso tiene

tres partes: quitar el símbolo, cambiar el guion por un nulo real y convertir la columna a número.

7.1 str.replace(): cambiar un texto por otro

str.replace('lo que busco', 'lo que pongo', regex=False) cambia cada aparición del texto buscado por el texto nuevo. Si pones una cadena vacía, el símbolo desaparece.

```
df["asistencia"] = df["asistencia"].str.replace("%", "", regex=False)
```

Con el ejemplo de la muestra, la salida es:

```
 0    91
1     -
2    78
3    86
4     -
5    95
Name: asistencia, dtype: str
```

Cómo leer el resultado: el signo % desaparece de todas las celdas. Los guiones siguen ahí, porque el reemplazo solo cambió el símbolo.

Errores frecuentes:

- Olvidar regex=False con un símbolo como el punto o el paréntesis. Con regex, el punto significa cualquier carácter.

Tu turno: quita el signo % de asistencia_pct.

7.2 pd.to_numeric(): convertir a número sin romper el código

pd.to_numeric(serie, errors='coerce') convierte cada valor a número. Si un valor no se puede convertir, lo deja como nulo en lugar de detener el programa.

```
df["asistencia"] = pd.to_numeric(df["asistencia"].replace("-", np.nan), errors="coerce")
```

Con el ejemplo de la muestra, la salida es:

```
 0    91.0
1     NaN
2    78.0
3    86.0
4     NaN
5    95.0
Name: asistencia, dtype: float64
```

Cómo leer el resultado: los guiones se convierten en NaN (np.nan, el valor nulo de NumPy que Pandas reconoce), que es un nulo real. Los números se conservan. Antes de la conversión, el guion era texto; después, es un valor faltante que puedes tratar en el paso 7.

Errores frecuentes:

- Usar errors='raise' (el valor por defecto) con columnas que tienen texto: el programa se detiene.
- Convertir sin reemplazar antes el guion: el guion se vuelve nulo, pero también pierdes el registro de que era un faltante disfrazado. Regístralo antes.

Tu turno: convierte asistencia_pct y horas_trabajo_semana a número. Antes de convertir, registra cuántos guiones encontraste.

8. Paso 4: errores tipográficos en categorías

8.1 str.strip() y str.lower(): normalizar el texto

str.strip() quita los espacios al inicio y al final de cada valor. str.lower() pasa el texto a minúsculas. Se encadenan en ese orden para que todas las variantes queden en la misma forma antes de unificarlas.

```
df["programa"] = df["programa"].str.strip().str.lower()
```

Con el ejemplo de la muestra, la salida es:

```
 0    ingeniería de sistemas
1                 psicologia
2                    derecho
3    ingeniería de sistemas
4                 psicologia
5                 enfermería
Name: programa, dtype: str
```

Cómo leer el resultado: el espacio al final de 'Ingeniería de Sistemas ' desaparece. Ahora el texto está en minúsculas, así que 'Psicologia' y 'psicologia' son iguales. Observa que la tilde sigue ahí: lower() no la quita.

Errores frecuentes:

- Pasar a minúsculas sin quitar espacios: el valor sigue siendo distinto.
- Olvidar asignar el resultado: el dataframe no cambia.

Tu turno: normaliza las columnas programa, jornada, genero y ciudad_procedencia con strip() y lower(). Luego revisa value_counts() para ver qué variantes quedan.

8.2 replace() con un diccionario: unificar variantes

replace({'valor viejo': 'valor nuevo'}) cambia cada valor de la izquierda por el de la derecha. Los valores que no aparecen en el diccionario se quedan igual.

```
df["programa"] = df["programa"].replace({"psicologia": "Psicología"})
```

Con el ejemplo de la muestra, la salida es:

```
 0    ingeniería de sistemas
1                Psicología
2                   derecho
3    ingeniería de sistemas
4                Psicología
5                enfermería
Name: programa, dtype: str
```

Cómo leer el resultado: psicologia se convierte en Psicología, con la forma correcta. Los valores que no están en el diccionario, como derecho, no cambian. Por eso el diccionario debe tener todas las variantes que encontraste.

Errores frecuentes:

- Escribir una clave con mayúsculas cuando la columna ya está en minúsculas: nunca coincide.
- Unificar dos valores que significan cosas distintas. Antes de hacerlo, pregunta si la diferencia tiene sentido para el análisis.

Tu turno: escribe el diccionario de variantes para programa y ciudad_procedencia. Al terminar, cuenta con value_counts() cuántos valores cambiaron.

9. Paso 5: fechas en dos formatos

Las fechas de este dataset aparecen en dos formatos: AAAA-MM-DD (por ejemplo, 2024-02-15) y DD/MM/AAAA (por ejemplo, 15/02/2024). Pandas no sabe cuál usar en cada fila. La solución es intentar cada formato por separado y juntar los resultados.

Los códigos de formato son:

- %Y: año de cuatro dígitos (2024).
- %m: mes con dos dígitos (02).
- %d: día con dos dígitos (15).

9.1 pd.to_datetime() con formato explícito

pd.to_datetime(serie, format='...', errors='coerce') convierte texto en fecha usando el formato que indiques. Si una celda no cumple ese formato, la deja como nulo (NaT, que es el nulo de las fechas) en lugar de detener el programa.

```
iso = pd.to_datetime(df["fecha"], format="%Y-%m-%d", errors="coerce")
dmy = pd.to_datetime(df["fecha"], format="%d/%m/%Y", errors="coerce")
df["fecha"] = iso.fillna(dmy)
```

Cómo leer el resultado: la primera línea convierte solo las fechas en formato ISO. Las que están en DD/MM quedan como nulo. La segunda línea hace lo contrario. La tercera rellena los nulos de la primera con los valores de la segunda. Así cada fila queda con una fecha válida.

Errores frecuentes:

- Usar un formato sin errors='coerce': el programa se detiene en la primera fecha que no coincide.
- Intentar un solo formato para todo: la mitad de las fechas quedan vacías.
- Confundir día y mes: 03/04/2023 puede ser 3 de abril o 4 de marzo. Por eso el formato debe ser explícito.

Tu turno: convierte fecha_matricula con los dos formatos. Antes de convertir, registra cuántas fechas estaban en DD/MM/AAAA.

Así se ve el proceso con tres fechas de ejemplo:

```
 0   2024-02-15
1          NaT
2          NaT
dtype: datetime64[us]
```

```
 0          NaT
1   2024-02-15
2   2023-04-03
dtype: datetime64[us]
```

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

```
 0   2024-02-15
1   2024-02-15
2   2023-04-03
dtype: datetime64[us]
```

Cómo leer el resultado: la primera salida solo convierte la fecha ISO, y las demás quedan como NaT. La segunda hace lo contrario. La tercera junta las dos, y todas tienen fecha válida. El tipo datetime64 confirma que ya puedes calcular con fechas, por ejemplo, saber cuánto tiempo pasó entre dos matrículas.

10. Paso 6: duplicados

10.1 drop_duplicates() y reset_index(): eliminar y renumerar

drop_duplicates() elimina las filas repetidas y conserva la primera aparición. reset_index(drop=True) vuelve a numerar las filas desde 0 y descarta el índice viejo.

```
df = df.drop_duplicates().reset_index(drop=True)
```

Con el ejemplo de la muestra, la salida es:

	id	programa	jornada	promedio	asistencia	trabaja	horas	estado
0	E1	Ingeniería de Sistemas	Diurna	4.2	91%	no	0	activo
1	E2	Psicologia	nocturna	3.1	-	sí	20	desertor
2	E3	Derecho	Diurna	45.0	78%	no	0	activo
3	E4	Ingeniería de Sistemas	Nocturna	NaN	86%	sí	35	desertor
4	E5	Enfermería	Fin de semana	3.8	95%	no	-	graduado

Cómo leer el resultado: la fila duplicada desapareció. Sin reset_index, el índice saltaría un número. Con drop=True, el índice viejo no se convierte en una columna nueva.

Errores frecuentes:

- Olvidar asignar el resultado a df.
- Olvidar reset_index: el índice queda con huecos, y eso puede causar errores en pasos posteriores.

Tu turno: elimina los duplicados del dataset y reinicia el índice. Anota cuántas filas eliminaste.

11. Paso 7: faltantes reales

Ahora que tienes los nulos identificados, decides qué hacer con cada uno. Las opciones principales son eliminar la fila, reemplazar el valor o dejarlo como nulo. La decisión depende del dato: no se inventa una nota, pero sí se puede marcar una ciudad como 'Sin dato'.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Un punto importante: no todos los nulos son errores. La columna motivo_retiro está vacía en todos los estudiantes que no desertaron, porque la pregunta no aplica para ellos. Ese nulo es esperado. Un nulo es un error solo cuando el dato debería existir y no existe.

11.1 dropna(subset=...): eliminar filas con nulos en columnas específicas

dropna(subset=['columna']) elimina las filas que tienen nulo en esa columna. Si no usas subset, elimina las filas con nulos en cualquier columna, incluso en columnas que no te importan.

```
df = df.dropna(subset=["promedio"])
```

Con el ejemplo de la muestra, la salida es:

```
   id                programa        jornada  promedio asistencia trabaja horas    estado
0  E1  Ingeniería de Sistemas         Diurna       4.2        91%      no     0    activo
1  E2              Psicologia       nocturna       3.1          -      sí    20  desertor
2  E3                 Derecho         Diurna      45.0        78%      no     0    activo
4  E2              Psicologia       nocturna       3.1          -      sí    20  desertor
5  E5              Enfermería  Fin de semana       3.8        95%      no     -  graduado
```

Cómo leer el resultado: la fila 3 desapareció porque su promedio es nulo. El índice salta el 3, por eso necesitas reset_index después.

Errores frecuentes:

- Sin subset, dropna elimina más de lo necesario: una fila con un nulo en una columna que no usas también se pierde.
- No asignar el resultado: la tabla no cambia.

Tu turno: decide qué hacer con los nulos de promedio_acumulado. ¿Puedes inventar una nota? Justifica tu decisión.

11.2 fillna(): reemplazar nulos por un valor

fillna(valor) reemplaza cada nulo por el valor que indiques. Para columnas de texto puedes usar una etiqueta como 'Sin dato'. Para columnas numéricas, la mediana es una opción común porque no se ve afectada por valores extremos.

```
df["edad"] = df["edad"].fillna(df["edad"].median())
```

Cómo leer el resultado: el nulo de edad se reemplaza por la mediana. Si la columna no se asigna de nuevo, el nulo sigue ahí.

Errores frecuentes:

- Reemplazar con 0 cualquier columna: un saldo de 0 y una edad de 0 significan cosas muy distintas.
- Calcular la mediana después de reemplazar los nulos: la mediana debe calcularse con los valores reales.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Tu turno: decide con qué valor reemplazarías los nulos de ciudad_procedencia y tipo_colegio. Justifica por qué.

12. Paso 8: valores imposibles

Los valores imposibles se detectan con condiciones (paso 6.5) y se corrigen con reglas que tienen sentido para el dato. Por ejemplo, la escala del promedio es de 0 a 5, y una semana tiene 168 horas. Antes de decidir, pregúntate si el valor es un error de registro o un dato real poco común.

12.1 Convertir valores imposibles en nulos y tratarlos

Primero marcas el valor como nulo con loc, y después lo tratas como cualquier otro nulo. Así la decisión queda separada de la detección.

```
df.loc[(df["trabaja"] == "no") & df["horas"].isnull(), "horas"] = 0
```

Con el ejemplo de la muestra, la salida es:

```
0     0.0
1    20.0
2     0.0
3    35.0
4    20.0
5     0.0
Name: horas, dtype: float64
```

Cómo leer el resultado: la condición marca las filas donde trabaja es 'no' y horas es nulo. A esas filas se les asigna 0 horas. La regla depende de otra columna, por eso no sirve un reemplazo general.

Errores frecuentes:

- Usar loc sin paréntesis en cada condición.
- Poner 0 a todos los nulos de una columna cuando la regla solo aplica a un grupo.

Tu turno: aplica la regla para horas_trabajo_semana: si el estudiante no trabaja, sus horas son 0. Después convierte en nulo cualquier valor mayor a 100 y reemplázalo por la mediana de quienes trabajan.

Mediana: el valor que queda justo en el centro cuando ordenas los datos. Se calcula con median().

```
10.0
```

Cómo leer el resultado: en la muestra, después de la regla de cero horas, tres de los seis valores son 0 y los otros tres son 20, 20 y 35. La mediana cae entre esos dos grupos, en 10. Por eso la mediana resume mejor la columna cuando hay valores extremos que el promedio.

Errores frecuentes en este paso: decidir una regla sin justificarla, y tratar un valor imposible como si fuera un dato real que solo está en otra escala.

13. Paso 9: variables nuevas

Una variable nueva se crea a partir de las que ya tienes. En este taller crearás dos: una columna desertor, con True o False, y una columna de cuartil de asistencia, que divide a los estudiantes en cuatro grupos de tamaño parecido.

13.1 eq(): comparar con un valor

df['columna'].eq('valor') compara cada fila con el valor indicado y devuelve True o False. Es lo mismo que usar ==, pero se encadena mejor con otros métodos.

```
df["desertor"] = df["estado"].eq("desertor")
```

Con el ejemplo de la muestra, la salida es:

```
 0    False
1     True
2    False
3     True
4     True
5    False
Name: estado, dtype: bool
```

Cómo leer el resultado: la columna desertor es True solo donde el estado es 'desertor'. Ahora puedes calcular proporciones: el promedio de True y False es la proporción de True.

Errores frecuentes:

- Comparar contra 'Desertor' con mayúscula cuando la columna tiene 'desertor'.

Tu turno: crea la columna desertor. Después cuenta cuántos True hay, y compara ese número con el total de estudiantes.

13.2 pd.qcut(): dividir en cuartiles

pd.qcut(serie, 4, labels=[...]) divide los valores en cuatro grupos con el mismo número de estudiantes. Los cortes se calculan a partir de los datos, así que cada grupo tiene aproximadamente la misma cantidad de filas.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

```
df["cuartil"] = pd.qcut(df["asistencia"], 4, labels=["Q1", "Q2", "Q3", "Q4"])
```

Con el ejemplo de la muestra, la salida es:

```
0    Q3
1    Q2
2    Q2
3    Q4
4    Q1
5    Q3
6    Q1
7    Q4
dtype: category
Categories (4, str): ['Q1' < 'Q2' < 'Q3' < 'Q4']
```

Cómo leer el resultado: cada grupo tiene el mismo número de estudiantes aproximadamente. Q1 es el grupo con la asistencia más baja y Q4 el más alta. En el análisis, observa si la deserción cambia de un grupo al siguiente.

Errores frecuentes:

- Usar cut() cuando quieres grupos de igual tamaño: cut() divide por rangos de igual longitud y los grupos pueden quedar muy desiguales.
- Olvidar que la columna debe ser numérica: si asistencia sigue como texto, qcut falla.

Tu turno: crea la columna cuartil_asistencia. Pregunta: ¿por qué conviene usar cuartiles en lugar de valores exactos de asistencia?

14. Gráficos

Los gráficos no se entregan solos: tienen que responder una pregunta sobre los datos. Cada gráfico de este taller corresponde a un paso del proceso.

14.1 Mapa de faltantes

Un mapa de faltantes dibuja una cuadrícula en la que cada columna es una franja vertical y cada fila del dataset es una línea horizontal. Las celdas vacías se marcan con un color, así que puedes ver de un vistazo qué columnas tienen huecos y si están concentrados en algunos estudiantes.

Código para dibujarlo:

```
sns.heatmap(df.isnull(), cbar=False)
```

```
plt.savefig(GRAFICOS / "nombre.png", bbox_inches="tight")
```

Pregunta para tu README: ¿qué columnas tienen los huecos más visibles? ¿Coinciden con los nulos que contaste en el paso 6.1?

14.2 Boxplots antes y después

Un boxplot resume una columna numérica. La caja cubre el 50 % central de los datos, la línea dentro de la caja es la mediana, los bigotes llegan a los valores normales y los puntos sueltos son posibles valores extremos. Comparar el boxplot antes y después de limpiar muestra el efecto de tus decisiones.

plt.subplots(2, 2) crea una cuadrícula de dos filas y dos columnas, con cuatro espacios para dibujar. Cada espacio se llama con ejes[fila, columna]: ejes[0, 0] es el superior izquierdo. Así puedes comparar antes y después en la misma figura.

```
fig, ejes = plt.subplots(2, 2, figsize=(12, 8))
sns.boxplot(x=df["promedio_acumulado"], ax=ejes[0, 0])
ejes[0, 0].set_title("promedio_acumulado (antes)")
```

```
sns.boxplot(x=df["columna"])
```

Pregunta para tu README: ¿qué cambió en cada distribución? ¿Algún valor extremo sigue ahí, y por qué decidiste dejarlo?

14.3 Tasa por grupo

Para calcular una tasa por grupo, usa groupby() con mean() sobre una columna de True y False. El resultado es una serie con un valor por grupo, que puedes graficar como barras con plot(kind='bar'). Cada barra es la proporción de desertores de ese grupo.

```
programa
Derecho                    0.0
Enfermería                 0.0
Ingeniería de Sistemas     0.0
Ingeniería de Sistemas     1.0
Psicologia                 1.0
Name: desertor, dtype: float64
```

Cómo leer el resultado: cada fila es un programa, y el número es la proporción de desertores. Un valor de 0.40 significa que cuatro de cada diez estudiantes del programa desertaron.

observed=True en groupby() le pide a Pandas que use solo las categorías que existen en los datos. Sin ese parámetro, las categorías de qcut que no tengan estudiantes aparecen con valor vacío y pueden confundir la lectura.

Errores frecuentes: comparar tasas de grupos con muy pocos estudiantes, porque una diferencia pequeña en número puede verse como una gran diferencia en proporción. Revisa siempre el tamaño de cada grupo.

Gráfico 3: tasa de deserción por programa. Gráfico 4: tasa de deserción por cuartil de asistencia. Ambos se dibujan con el mismo patrón: calcular la tasa y graficarla como barras.

```
tasa = df.groupby("programa")["desertor"].mean()
tasa.plot(kind="bar")
plt.title("Tasa de deserción por programa")
```

Pregunta para tu README: ¿qué programa tiene la tasa más alta? ¿Hay un programa con pocos estudiantes que deba verse con cuidado?

15. Paso 10: las cinco preguntas

Las cinco preguntas se responden con groupby() y agg(). Cada respuesta debe incluir el número de estudiantes por grupo, porque sin ese dato una tasa no se puede interpretar.

15.1 groupby() con agg(): varios resúmenes a la vez

groupby('columna') separa las filas en grupos. agg() calcula varios resúmenes por grupo y les da nombre: tasa es el promedio de desertor, y estudiantes es el número de filas de cada grupo.

```
df.groupby("jornada").agg(tasa=("desertor", "mean"), estudiantes=("desertor", "size"))
```

Con el ejemplo de la muestra, la salida es:

jornada	tasa	estudiantes
Diurna	0.0	2
Fin de semana	0.0	1
Nocturna	1.0	1
nocturna	1.0	2

Cómo leer el resultado: cada fila es un grupo de jornada. La tasa muestra la proporción de desertores y estudiantes muestra el tamaño del grupo. En la muestra, con dos estudiantes por jornada, cualquier diferencia es muy inestable; en el dataset real los grupos son grandes.

Errores frecuentes:

- Escribir agg('mean') sin nombres: el resultado no tiene columnas claras.
- Olvidar el tamaño del grupo al interpretar la tasa.

Tu turno: usa groupby() y agg() para responder la pregunta 1 (programa con mayor deserción), y repite el cálculo para la pregunta 2 (beca).

15.2 Tasa por grupo con mean() sobre True y False

Cuando una columna es True y False, su promedio es la proporción de True. Así se calcula la tasa de deserción por grupo sin escribir operaciones adicionales.

```
print(df.groupby("programa")["desertor"].mean())
```

Con el ejemplo de la muestra, la salida es:

```
programa
Derecho                   0.0
Enfermería                0.0
Ingeniería de Sistemas    0.0
Ingeniería de Sistemas    1.0
Psicologia                1.0
Name: desertor, dtype: float64
```

Cómo leer el resultado: el resultado es una proporción por programa. Un valor de 1.0 significa que todos los estudiantes del grupo son desertores, y 0.0 que ninguno lo es. En la muestra, los grupos son tan pequeños que los extremos no sirven para concluir nada.

Errores frecuentes:

- Usar sum() cuando quieres una proporción: sum() da la cantidad, mean() da la proporción.

Tu turno: responde la pregunta 2 (becados frente a no becados) con esta técnica.

15.3 Las cinco preguntas

Pregunta 1: ¿qué programa presenta la tasa de deserción más alta? Calcula la tasa por programa, muestra el número de estudiantes en cada uno y responde con esos dos datos.

Pregunta 2: ¿hay diferencia en la deserción entre becados y no becados? Compara las dos tasas. Después escribe, con tus palabras, por qué una diferencia observada no prueba que la beca cause la permanencia.

Pregunta 3: ¿qué jornada tiene el menor promedio acumulado? Calcula el promedio de promedio_acumulado por jornada. Revisa también cuántos estudiantes hay en cada jornada.

Pregunta 4: ¿cuántas materias reprobadas tienen en promedio los desertores frente a los activos? Excluye a los graduados antes de calcular, porque no pertenecen a ninguno de los dos grupos de la comparación. Escribe por qué la exclusión es necesaria.

Pregunta 5: ¿la deserción aumenta conforme baja la asistencia? Divide la asistencia en cuartiles con pd.qcut(), calcula la tasa de deserción en cada cuartil y muestra la asistencia promedio de cada grupo. Describe el patrón con tus palabras y explica por qué no es una prueba de causa.

16. Paso 11: guardar resultados

Al terminar los pasos, el script guarda dos archivos. El primero es el dataset limpio, que contiene todas las columnas originales y las nuevas, sin duplicados ni errores tipográficos. El segundo es la tabla de decisiones.

```
df.to_csv(PROCESSED / "dataset_limpio.csv", index=False)
pd.DataFrame(DECISIONES).to_csv(PROCESSED / "decisiones.csv", index=False)
```

El parámetro index=False evita que la numeración de filas se guarde como una columna extra. Sin él, cada vez que leas el archivo aparecería una columna sin nombre con los números de fila.

Antes de dar por terminado el taller, revisa que existan Data/processed/dataset_limpio.csv, Data/processed/decisiones.csv y los cuatro gráficos en Data/processed/graficos/.

17. Reflexión

Responde estas tres preguntas en tu README. No hay una respuesta única; se evalúa el razonamiento.

Reflexión 1: asociación y causalidad.

Si la deserción es menor entre los becados, ¿la beca causa la permanencia? Piensa en otras razones por las que un estudiante podría recibir la beca y también permanecer, y en cómo se distinguiría una causa de una asociación.

Reflexión 2: variables que no deberían usarse para predecir.

La columna motivo_retiro solo se conoce después de que el estudiante deserta. ¿Puede usarse para predecir la deserción antes de que ocurra? ¿Qué otras columnas podrían tener el mismo problema?

Reflexión 3: variables sensibles. Género, estrato y ciudad de procedencia pueden reflejar desigualdades sociales.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Si una de estas variables aparece asociada a la deserción, ¿debería la institución usarla para decidir qué estudiantes reciben apoyo? ¿Qué riesgos tiene?

18. Entrega

Sube tu código completo al repositorio y envía el enlace por el formulario de entrega. La entrega es individual.

El repositorio debe incluir:

- analisis_desercion.py con todos los TODO completados.
- Data/processed/decisiones.csv y la carpeta Data/processed/graficos/ con los cuatro gráficos.
- El diccionario de datos completado en el README.
- Un README.md con una sección titulada Mis decisiones y respuestas, que incluya: tu diagnóstico de calidad, tus decisiones con su justificación, las respuestas a las cinco preguntas con el número de estudiantes por grupo, y tus respuestas a las tres reflexiones.

El script debe ejecutarse sin errores desde cero. Cada decisión debe tener una justificación, y cada conclusión debe apoyarse en un número del análisis.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net